

Progress Report 1

Project: Real-Time Market Volatility and Trade Activity Tracking

Student: Michael Hein (300375535)

Course: CSIS 4495 – 003

Instructor: Padmapriya Arasanipalai Kandhadai

Reporting Period: Proposal (Jan 27) to Week 5 (Feb 8)

Submission Date: Feb 9, 2026

Overview

Since the project proposal, the primary focus has been the transition from theoretical design to a functional **Event-Driven Architecture (EDA)**. Key achievements include the orchestration of a containerized Kafka environment via **Docker Compose**, the establishment of a real-time ingestion link with the **Coinbase WebSocket API**, and the implementation of a **PySpark Structured Streaming** consumer capable of handling high-velocity market data.

Detailed Work Log

Week 4 (Jan 27 – Jan 30): Infrastructure & Environment Orchestration

- **Wed, Jan 29 (4 hrs):**
 - Refined project methodology to utilize **Amazon Redshift** as the final persistence layer.
 - Designed the JSON schema for incoming trade events including price, size, and product_id.
- **Thu, Jan 30 (2 hrs):**
 - Configured local development environment on macOS.
 - Initialized Git repository and established branching strategies for producer and consumer logic.

Week 5 (Feb 3 – Feb 8): Stream Ingestion & Processing Logic

- **Mon, Feb 3 (4 hrs):**

- Drafted `docker-compose.yml` to orchestrate **Zookeeper** and **Kafka** brokers.
 - Configured internal/external listeners for seamless communication between Docker containers and the local PySpark environment.
 - **Tue, Feb 4 (5 hrs):**
 - Developed a Python-based Kafka Producer integrated with the **Coinbase WebSocket** to stream live BTC-USD trade data into the `crypto.trades.raw` topic.
 - Implemented exception handling for API connection drops.
 - **Wed, Feb 5 (5 hrs):**
 - Initialized the **SparkSession** with required Kafka-SQL dependencies.
 - Developed the PySpark Structured Streaming consumer to read from the broker using the `subscribe` pattern.
 - Utilized `from_json()` with a predefined `StructType` schema to parse raw string values into typed columns.
 - **Thu, Feb 6 (4 hrs):**
 - Conducted end-to-end integration tests.
 - Optimized Spark trigger intervals and stabilized the connection between Kafka and Spark.
-

Current Status

- **Data Ingestion:** Fully operational; Kafka producer successfully streams live trade events to the broker.
 - **Stream Processing:** PySpark engine is active and correctly deserializing incoming streams into a structured format.
 - **Visualization:** Initial Streamlit dashboard is connected and capable of displaying live metrics from the stream.
-

Planned Next Steps (Week 6)

- **Feature Engineering:** Implement `window()` functions for **Volume Weighted Average Price (VWAP)** and rolling volatility.
- **State Management:** Configure **watermarking** to handle out-of-order events and improve error handling.
- **Persistence:** Integrate the **Amazon Redshift** storage sink to transition from live-only data to persistent storage.